



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

**Escola Superior d'Enginyeries Industrial,
Aeroespacial i Audiovisual de Terrassa**

Bachelor's Degree in Aerospace Vehicle Engineering

Class Work

Mathematics and Computing Engineering

Marina Liao Colom Escolà

June 2, 2026

Contents

1	Image Compression C Program	2
1.1	Introduction and Objectives	2
1.2	Background Theory	2
1.2.1	Lossless Image Compression	2
1.2.2	Run-Length Encoding (RLE)	2
1.3	Program Design	3
1.3.1	Image Loading	3
1.3.2	Compression Function	3
1.3.3	Decompression Function	4
1.4	Complexity Analysis	4
1.5	Compilation and Execution	4
1.6	Results	5
1.7	Conclusion	6
2	RSA and ECC Public Key Cryptography C Program	7
2.1	Introduction and Objectives	7
2.2	Background Theory	7
2.2.1	RSA (Rivest–Shamir–Adleman)	7
2.2.2	ECC (Elliptic Curve Cryptography)	8
2.3	Program Design	8
2.3.1	RSA Module	8
2.3.2	ECC Module	9
2.4	Complexity Analysis	9
2.5	Compilation and Execution	9
2.6	Results	10
2.7	Conclusion	10
3	Bibliography	11
A	Image Compression C Code	12
B	RSA and ECC Public Key C Code	16

1 Image Compression C Program

1.1 Introduction and Objectives

Digital images require large amounts of storage space, particularly high-resolution images. Image compression reduces the amount of data required to represent an image while maintaining acceptable image quality.

Image compression techniques are broadly divided into two categories [Gonzalez and Woods, 2018]:

- Lossless Compression
- Lossy Compression

Lossless compression preserves every bit of information in the original image, allowing exact reconstruction after decompression. Lossy compression removes some image information to achieve higher compression ratios.

This task focuses on lossless image compression using the Run-Length Encoding (RLE) algorithm [Salomon, 2004]. The implementation has been developed in the C programming language and supports both grayscale and RGB color images.

The primary objectives of this task are:

- To design a lossless image compression system.
- To implement Run-Length Encoding in C.
- To support grayscale and color images.
- To implement decompression for exact image recovery.
- To analyze compression efficiency and computational complexity.

1.2 Background Theory

1.2.1 Lossless Image Compression

Lossless compression is a process where image data can be reconstructed perfectly after decompression.

Examples include [Gonzalez and Woods, 2018]:

- Run-Length Encoding (RLE)
- Huffman Coding
- Lempel-Ziv-Welch (LZW)
- PNG compression

The major advantage of lossless compression is that no image information is lost.

1.2.2 Run-Length Encoding (RLE)

Run-Length Encoding is a simple lossless compression technique that replaces repeated sequences of data values with a count and a single value [Nelson and Gailly, 1991].

For example:

1111111222222333

can be represented as:

$(7, 1)(6, 2)(3, 3)$

where:

- First value = repetition count
- Second value = pixel value

The RLE algorithm works particularly well for images containing repeated pixel regions, flat colors, and simple graphics.

1.3 Program Design

The project contains several major functions.

1.3.1 Image Loading

The program reads:

- PGM grayscale images (P5)
- PPM color images (P6)

The image metadata includes:

- Width
- Height
- Maximum intensity value
- Number of channels

1.3.2 Compression Function

The compression function:

- scans pixel values,
- counts consecutive repetitions,
- stores count-value pairs.

The compression workflow is:

1. Read image file.
2. Detect image format (P5 or P6).
3. Load image pixels into memory.
4. Scan pixel values sequentially.
5. Count repeated consecutive values.
6. Store:
 - repetition count
 - pixel value
7. Save compressed output.

1.3.3 Decompression Function

The decompression function reconstructs the original image using stored count-value pairs. Since RLE is lossless, the reconstructed image is identical to the original image.

The decompression workflow is:

1. Read compressed file.
2. Recover metadata:
 - width
 - height
 - channels
 - maximum pixel value
3. Read RLE pairs.
4. Reconstruct original pixels.
5. Save decompressed image.

1.4 Complexity Analysis

Let n represent the total number of pixels.

The time complexity of compression is $O(n)$ because every pixel is visited once.

The decompression complexity is also $O(n)$ since each pixel is reconstructed exactly once.

The memory complexity is $O(n)$ because image pixels are loaded into memory.

1.5 Compilation and Execution

Compile using GCC [gcc, 2026]:

```
1 gcc ImageCompressor.c -o program
```

Compress grayscale images with:

```
1 ./program c image.pgm compressed.rle
```

Compress RGB color images with:

```
1 ./program c image.ppm compressed.rle
```

Decompress using:

```
1 ./program d compressed.rle restored.ppm
```

or

```
1 ./program d compressed.rle restored.pgm
```

1.6 Results

The implemented system (see code in Appendix A) successfully:

- compressed grayscale images,
- compressed RGB color images,
- decompressed files correctly,
- reconstructed the original image exactly.

The reconstructed images (Figures 3 and 4) matched the original images (Figures 1 and 2) without any loss of information.

Run-Length Encoding (RLE) is a simple and computationally efficient lossless compression technique. Since it processes image data sequentially, both compression and decompression are performed quickly while preserving the original image without information loss.

The effectiveness of RLE strongly depends on image characteristics. Images containing large uniform regions, repeated colors, or simple graphical structures generally achieve better compression because consecutive pixel values are often repeated.

However, RLE performs poorly on noisy or highly detailed images, where neighboring pixel values vary significantly. In such cases, compression efficiency decreases and the compression ratio becomes limited compared to more advanced methods. Therefore, RLE is most suitable for structured images rather than natural photographs with complex textures.



Figure 1: Original RGB color image



Figure 2: Original grayscale image



Figure 3: Reconstructed RGB color image



Figure 4: Reconstructed grayscale image

1.7 Conclusion

This task presented a lossless image compression system using Run-Length Encoding implemented in the C programming language. The proposed system successfully compressed and decompressed grayscale and color images while preserving exact image quality.

The project demonstrated that RLE is computationally efficient and easy to implement, making it suitable for educational purposes and images containing repeated patterns. However, its performance depends heavily on image content and is less effective for complex natural scenes.

2 RSA and ECC Public Key Cryptography C Program

2.1 Introduction and Objectives

Public-key cryptography is a cornerstone of modern secure communication. Unlike symmetric cryptography, it uses a pair of keys: a public key for encryption and a private key for decryption [Stallings, 2017].

Two widely used public-key systems are:

- RSA (Rivest–Shamir–Adleman)
- ECC (Elliptic Curve Cryptography)

This task implements simplified versions of both algorithms to illustrate their mathematical foundations.

The main objective of this project is to implement and analyze two public-key cryptographic systems, RSA and Elliptic Curve Cryptography (ECC), using the C programming language. The task aims to demonstrate the fundamental concepts of asymmetric encryption through simplified educational implementations.

The specific objectives are:

- To understand the mathematical foundations of RSA and ECC.
- To implement RSA key generation, encryption, and decryption in C.
- To implement basic elliptic curve operations used in ECC.
- To demonstrate encryption and decryption processes using ECC.
- To compare the characteristics and computational aspects of RSA and ECC.
- To provide an educational implementation that illustrates the principles of public-key cryptography.

2.2 Background Theory

2.2.1 RSA (Rivest–Shamir–Adleman)

RSA is based on the difficulty of factoring large integers [Rivest et al., 1978]. The key generation process involves:

$$n = p \cdot q \tag{1}$$

$$\phi(n) = (p - 1)(q - 1) \tag{2}$$

A public key (e, n) is chosen such that:

$$\gcd(e, \phi(n)) = 1$$

The private key d satisfies:

$$d \equiv e^{-1} \pmod{\phi(n)}$$

The encryption consists in:

$$C = M^e \pmod{n}$$

And the decryption in:

$$M = C^d \pmod{n}$$

2.2.2 ECC (Elliptic Curve Cryptography)

ECC is based on elliptic curves defined as:

$$y^2 = x^3 + ax + b \pmod{p}$$

where p is a prime number defining a finite field [Certicom Research, 2009].

It relies on two operations:

- Point addition

If $P \neq Q$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}$$

If $P = Q$:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}$$

- Scalar multiplication

$$kP = P + P + \dots + P \quad (k \text{ times})$$

The public key is expressed as: $Q = kG$

The Ciphertext as: (C_1, C_2)

$$C_1 = rG, \quad C_2 = M + rQ$$

And the decryption as:

$$M = C_2 - kC_1$$

2.3 Program Design

The program has been designed as an educational implementation of two public-key cryptographic systems: RSA and Elliptic Curve Cryptography (ECC). The implementation is divided into independent modules to improve readability and simplify the understanding of the algorithms.

2.3.1 RSA Module

The RSA module is responsible for key generation, encryption, and decryption. First, two prime numbers are selected to compute the modulus n and Euler's totient function $\phi(n)$. A public exponent e is chosen such that it is relatively prime to $\phi(n)$, and a private key d is computed as the modular inverse of e .

The RSA workflow consists of:

- Prime number selection
- Public and private key generation
- Message encryption
- Message decryption

The encryption and decryption processes use modular exponentiation to efficiently compute powers under modular arithmetic.

2.3.2 ECC Module

The ECC module implements elliptic curve operations over a finite field. The curve follows the general equation:

$$y^2 = x^3 + ax + b \pmod{p}$$

The implementation includes point addition and scalar multiplication, which are fundamental operations for elliptic curve cryptography.

The ECC workflow consists of:

- Definition of elliptic curve parameters
- Generator point selection
- Private and public key generation
- Message encryption using point operations
- Message decryption through inverse point computation

The program is organized into several functions responsible for specific tasks:

- Mathematical utility functions (GCD, modular exponentiation, modular inverse)
- RSA functions for key generation and encryption/decryption
- ECC functions for point addition and scalar multiplication
- Demonstration functions to display encryption and decryption results

This modular structure improves code clarity and facilitates the understanding of both cryptographic systems.

2.4 Complexity Analysis

In this case, for RSA:

- Key generation: $O(\sqrt{n})$
- Encryption: $O(\log e)$
- Decryption: $O(\log d)$

For ECC

- Scalar multiplication: $O(k)$
- Point addition: $O(1)$

2.5 Compilation and Execution

Compile using GCC [gcc, 2026]:

```
1 gcc RSA_ECC_PublicKey.c -o crypto
```

Then execute the program using:

```
1 ./crypto
```

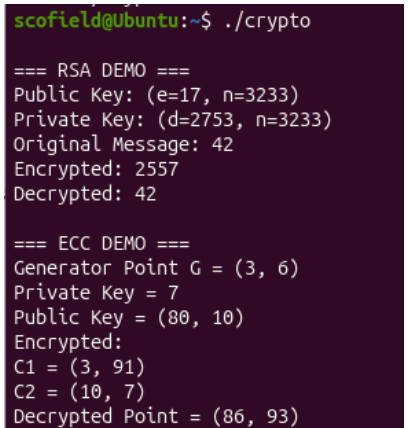
The program will display the generated RSA keys, encrypted and decrypted messages, as well as the ECC public key generation and encryption/decryption demonstration.

2.6 Results

The program (Appendix B) successfully demonstrates (Figure 5):

- RSA encryption and decryption using modular arithmetic
- ECC point operations over a finite field
- Secure message transformation and recovery

Although simplified, the implementation correctly illustrates the mathematical structure of both cryptosystems.



```
scofield@ubuntu:~$ ./crypto

=== RSA DEMO ===
Public Key: (e=17, n=3233)
Private Key: (d=2753, n=3233)
Original Message: 42
Encrypted: 2557
Decrypted: 42

=== ECC DEMO ===
Generator Point G = (3, 6)
Private Key = 7
Public Key = (80, 10)
Encrypted:
C1 = (3, 91)
C2 = (10, 7)
Decrypted Point = (86, 93)
```

Figure 5: Output for the RSA ECC Public Key code

This implementation is intended for educational purposes only:

- Uses small integer values (not secure)
- No optimized prime generation
- ECC uses a toy curve
- No padding schemes (e.g., OAEP)

2.7 Conclusion

This task provided a practical demonstration of RSA and ECC cryptographic systems using the C programming language. The implementation highlights the mathematical principles behind public-key cryptography and shows how encryption and decryption are performed at a fundamental level.

ECC provides better security per key size compared to RSA, making it widely used in modern cryptographic applications.

All files related to these two tasks can be consulted at Marina Colom's GitHub page [(scofield), 2026].

3 Bibliography

References

- [gcc, 2026] (2026). *GNU Compiler Collection (GCC) Documentation*. GNU Project. Available online.
- [Certicom Research, 2009] Certicom Research (2009). Sec 1: Elliptic curve cryptography. Standards for Efficient Cryptography Group (SECG).
- [Gonzalez and Woods, 2018] Gonzalez, R. C. and Woods, R. E. (2018). *Digital Image Processing*. Pearson, 4 edition.
- [Nelson and Gailly, 1991] Nelson, M. and Gailly, J.-L. (1991). *The Data Compression Book*. M&T Books.
- [Rivest et al., 1978] Rivest, R. L., Shamir, A., and Adleman, L. M. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126.
- [Salomon, 2004] Salomon, D. (2004). *Data Compression: The Complete Reference*. Springer, New York, 3 edition.
- [(sc0ffield), 2026] (sc0ffield), M. C. (2026). 220127-mathematics-and-computing-engineering: Classwork. <https://github.com/sc0ffield/220127-Mathematics-and-Computing-Engineering/tree/main/ClassWork>. GitHub repository directory, accessed 2026-06-02.
- [Stallings, 2017] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson, 7 edition.

A Image Compression C Code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX_LINE 256
6
7  typedef struct {
8      int width;
9      int height;
10     int maxValue;
11     int channels;
12     unsigned char *data;
13 } Image;
14
15 void freeImage(Image *img) {
16     if (img->data != NULL) {
17         free(img->data);
18         img->data = NULL;
19     }
20 }
21
22 int loadImage(const char *filename, Image *img) {
23     FILE *file = fopen(filename, "rb");
24
25     if (!file) {
26         printf("Error: Cannot open file %s\n", filename);
27         return 0;
28     }
29
30     char format[3];
31
32     if (fscanf(file, "%2s", format) != 1) {
33         fclose(file);
34         return 0;
35     }
36
37     if (strcmp(format, "P5") == 0) {
38         img->channels = 1;
39     } else if (strcmp(format, "P6") == 0) {
40         img->channels = 3;
41     } else {
42         printf("Unsupported format. Use PGM (P5) or PPM (P6).\n");
43         fclose(file);
44         return 0;
45     }
46
47     int c = fgetc(file);
48
49     while ((c = fgetc(file)) == '#') {
50         while (fgetc(file) != '\n');
51     }
52
53     ungetc(c, file);
54
55     fscanf(file, "%d %d", &img->width, &img->height);
56     fscanf(file, "%d", &img->maxValue);
57     fgetc(file);
58
59     int imageSize = img->width * img->height * img->channels;
60
61     img->data = (unsigned char *)malloc(imageSize);
62
63     if (!img->data) {
64         printf("Memory allocation failed.\n");
65         fclose(file);
```

```
66     return 0;
67 }
68
69 fread(img->data, 1, imageSize, file);
70 fclose(file);
71
72 return 1;
73 }
74
75 int saveImage(const char *filename, Image *img) {
76     FILE *file = fopen(filename, "wb");
77
78     if (!file) {
79         printf("Cannot save image.\n");
80         return 0;
81     }
82
83     if (img->channels == 1) {
84         fprintf(file, "P5\n");
85     } else {
86         fprintf(file, "P6\n");
87     }
88
89     fprintf(file, "%d %d\n", img->width, img->height);
90     fprintf(file, "%d\n", img->maxValue);
91
92     int imageSize = img->width * img->height * img->channels;
93
94     fwrite(img->data, 1, imageSize, file);
95
96     fclose(file);
97
98     return 1;
99 }
100
101 int compressImage(const char *inputFile, const char *outputFile) {
102     Image img = {0};
103
104     if (!loadImage(inputFile, &img)) {
105         return 0;
106     }
107
108     FILE *out = fopen(outputFile, "wb");
109
110     if (!out) {
111         printf("Cannot create compressed file.\n");
112         freeImage(&img);
113         return 0;
114     }
115
116     fwrite(&img.width, sizeof(int), 1, out);
117     fwrite(&img.height, sizeof(int), 1, out);
118     fwrite(&img.maxValue, sizeof(int), 1, out);
119     fwrite(&img.channels, sizeof(int), 1, out);
120
121     int imageSize = img.width * img.height * img.channels;
122
123     int i = 0;
124
125     while (i < imageSize) {
126         unsigned char current = img.data[i];
127         unsigned int count = 1;
128
129         while (i + count < imageSize &&
130             img.data[i + count] == current &&
131             count < 4294967295U) {
132             count++;
133         }
```

```
134     fwrite(&count, sizeof(unsigned int), 1, out);
135     fwrite(&current, sizeof(unsigned char), 1, out);
136
137     i += count;
138 }
139
140 fclose(out);
141
142 printf("Compression complete.\n");
143
144 freeImage(&img);
145
146 return 1;
147 }
148
149 int decompressImage(const char *compressedFile,
150                    const char *outputImage) {
151
152     FILE *in = fopen(compressedFile, "rb");
153
154     if (!in) {
155         printf("Cannot open compressed file.\n");
156         return 0;
157     }
158
159     Image img = {0};
160
161     fread(&img.width, sizeof(int), 1, in);
162     fread(&img.height, sizeof(int), 1, in);
163     fread(&img.maxValue, sizeof(int), 1, in);
164     fread(&img.channels, sizeof(int), 1, in);
165
166     int imageSize =
167         img.width * img.height * img.channels;
168
169     img.data =
170         (unsigned char *)malloc(imageSize);
171
172     if (!img.data) {
173         fclose(in);
174         return 0;
175     }
176
177     int index = 0;
178
179     while (index < imageSize) {
180         unsigned int count;
181         unsigned char value;
182
183         fread(&count, sizeof(unsigned int), 1, in);
184         fread(&value, sizeof(unsigned char), 1, in);
185
186         for (unsigned int i = 0;
187              i < count && index < imageSize;
188              i++) {
189             img.data[index++] = value;
190         }
191     }
192
193     fclose(in);
194
195     if (!saveImage(outputImage, &img)) {
196         freeImage(&img);
197         return 0;
198     }
199
200     freeImage(&img);
201 }
```

```
202     printf("Decompression complete.\n");
203
204     return 1;
205 }
206
207 void printUsage() {
208     printf("Usage:\n");
209     printf("Compress:\n");
210     printf("./program c input.ppm output.rle\n\n");
211
212     printf("Decompress:\n");
213     printf("./program d input.rle output.ppm\n");
214 }
215
216 int main(int argc, char *argv[]) {
217
218     if (argc != 4) {
219         printUsage();
220         return 1;
221     }
222
223     char mode = argv[1][0];
224
225     if (mode == 'c') {
226         compressImage(argv[2], argv[3]);
227     }
228     else if (mode == 'd') {
229         decompressImage(argv[2], argv[3]);
230     }
231     else {
232         printUsage();
233     }
234
235     return 0;
236 }
237
```


B RSA and ECC Public Key C Code

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef long long ll;
5
6  ll gcd(ll a, ll b) {
7      while (b != 0) {
8          ll t = b;
9          b = a % b;
10         a = t;
11     }
12     return a;
13 }
14
15 ll modExp(ll base, ll exp, ll mod) {
16     ll result = 1;
17     base %= mod;
18
19     while (exp > 0) {
20         if (exp % 2 == 1)
21             result = (result * base) % mod;
22
23         base = (base * base) % mod;
24         exp /= 2;
25     }
26
27     return result;
28 }
29
30 void rsa_demo() {
31
32     printf("\n=== RSA DEMO ===\n");
33
34     ll p = 61;
35     ll q = 53;
36
37     ll n = p * q;
38     ll phi = (p - 1) * (q - 1);
39
40     ll e = 17;
41
42     if (gcd(e, phi) != 1) {
43         printf("Invalid e\n");
44         return;
45     }
46
47     ll d = 0;
48
49     for (ll i = 1; i < phi; i++) {
50         if ((e * i) % phi == 1) {
51             d = i;
52             break;
53         }
54     }
55
56     printf("Public Key: (e=%lld, n=%lld)\n", e, n);
57     printf("Private Key: (d=%lld, n=%lld)\n", d, n);
58
59     ll message = 42;
60
61     ll encrypted = modExp(message, e, n);
62     ll decrypted = modExp(encrypted, d, n);
63
64     printf("Original Message: %lld\n", message);
65     printf("Encrypted: %lld\n", encrypted);
```

```
66     printf("Decrypted: %lld\n", decrypted);
67 }
68
69 typedef struct {
70     ll x;
71     ll y;
72     int infinity;
73 } Point;
74
75 ll ec_a = 2;
76 ll ec_b = 3;
77 ll ec_p = 97;
78
79 ll modInv(ll a, ll p) {
80     a %= p;
81
82     for (ll i = 1; i < p; i++) {
83         if ((a * i) % p == 1)
84             return i;
85     }
86
87     return -1;
88 }
89
90 Point pointAdd(Point P, Point Q) {
91
92     Point R;
93     R.infinity = 0;
94
95     if (P.infinity)
96         return Q;
97
98     if (Q.infinity)
99         return P;
100
101     if (P.x == Q.x && P.y != Q.y) {
102         R.infinity = 1;
103         return R;
104     }
105
106     ll m;
107
108     if (P.x == Q.x) {
109         ll num = (3 * P.x * P.x + ec_a) % ec_p;
110         ll den = modInv(2 * P.y, ec_p);
111         m = (num * den) % ec_p;
112     } else {
113         ll num = (Q.y - P.y + ec_p) % ec_p;
114         ll den = modInv((Q.x - P.x + ec_p) % ec_p, ec_p);
115         m = (num * den) % ec_p;
116     }
117
118     R.x = (m * m - P.x - Q.x) % ec_p;
119     R.x = (R.x + ec_p) % ec_p;
120
121     R.y = (m * (P.x - R.x) - P.y) % ec_p;
122     R.y = (R.y + ec_p) % ec_p;
123
124     return R;
125 }
126
127 Point scalarMult(Point P, ll k) {
128
129     Point R;
130     R.infinity = 1;
131
132     while (k > 0) {
133
```

```
134         if (k % 2 == 1)
135             R = pointAdd(R, P);
136
137         P = pointAdd(P, P);
138         k /= 2;
139     }
140
141     return R;
142 }
143
144 void ecc_demo() {
145
146     printf("\n=== ECC DEMO ===\n");
147
148     Point G = {3, 6, 0};
149
150     ll privateKey = 7;
151
152     Point publicKey = scalarMult(G, privateKey);
153
154     printf("Generator Point G = (%lld, %lld)\n", G.x, G.y);
155     printf("Private Key = %lld\n", privateKey);
156     printf("Public Key = (%lld, %lld)\n", publicKey.x, publicKey.y);
157
158     Point M = {10, 7, 0};
159
160     ll k = 5;
161
162     Point C1 = scalarMult(G, k);
163     Point C2 = pointAdd(M, scalarMult(publicKey, k));
164
165     printf("Encrypted:\n");
166     printf("C1 = (%lld, %lld)\n", C1.x, C1.y);
167     printf("C2 = (%lld, %lld)\n", C2.x, C2.y);
168
169     Point S = scalarMult(C1, privateKey);
170
171     Point negS = {S.x, (ec_p - S.y) % ec_p, 0};
172
173     Point decrypted = pointAdd(C2, negS);
174
175     printf("Decrypted Point = (%lld, %lld)\n",
176           decrypted.x,
177           decrypted.y);
178 }
179
180 int main() {
181
182     rsa_demo();
183     ecc_demo();
184
185     return 0;
186 }
```